Singular Value Decomposition

Overview

- eigenvalues, and eigenvectors, and higher-dimensional tensors
- **convolution in 2-D as a linear transformation**
- convolution and eigen decomposition
- **SVD decomposition**
- left and right singular values
- **computing and utilizing numerical libraries**
- similarity/relationship between SVD and matrix diagonalization
- **computing rank-k approximations**
- using the Frobenius norm to compute distances in matrix space

Async

Eigenvalues for 2D Convolution

Convolution: 2D

- this is very similar to the 1-D case

$$v_{j_1,j_2}^{k_1,k_2} = \exp!\left(\underbrace{\frac{i2\pi j_1 k_1}{N_1}}_{\text{freq in x direction}}\right)\exp!\left(\underbrace{\frac{i2\pi j_2 k_2}{N_2}}_{\text{freq in y direction}}\right)$$

- ex: the **LaPlace** kernel

$$-u_{i,j-1} - u_{i-1,j} + 4u_{i,j} - u_{i+1,j} - u_{i,j+1}$$

eigenvectors for any convolutional kernel

each term is just a shift in the i or j coordinate

Numerical

$$Q\Lambda Q^H u$$

- for an $m \times n$ $2-D$ $array$, each matrix is $MN \times MN$
- **Fourier** mode

$$v_{j_1,j_2}^{k_1,k_2} = \exp!\left(\frac{i2\pi j_1 k_1}{N_1}\right)\exp!\left(\frac{i2\pi j_2 k_2}{N_2}\right)$$

one of $M \times N$ vectors in the Q $matrix$

- **however,** if M and N are powers of 2, the **cost** of the **FFT** is

$$O\big(MN(\log(M)+\log(N))\big) \quad \text{and not} \quad O((MN)^2)$$

Julia

Steps

- first, set up a **simple** u where we **know** the answer 1
- create the **Laplace stencil** on a **periodic** domain 2
- **evaluate** the Fourier **transform** of the **stencil** 3
  note that the values are all real numbers
- these correspond to the **eigenvalues**, but they are packed into an array **rather than** listed individually
- the **top-left** entry is 0, because that is the **eigenvalue** corresponding to the **constant** function 1

```
julia> using FFTW

julia> u = zeros(6,6); u[3,4] = 1;
6×6 Matrix{Float64}:
 0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  1.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0  0.0

julia> s=zeros(6,6); s[1,1]=4; s[2,1]=-1; s[1,2]=-1;
s[6,1]=-1; s[1,6]=1;
6×6 Matrix{Float64}:
 4.0  -1.0  0.0  0.0  0.0  -1.0
 -1.0  0.0  0.0  0.0  0.0   0.0
 0.0   0.0  0.0  0.0  0.0   0.0
 0.0   0.0  0.0  0.0  0.0   0.0
 0.0   0.0  0.0  0.0  0.0   0.0
 -1.0  0.0  0.0  0.0  0.0   0.0

julia> L = fft(s)
6×6 Matrix{ComplexF64}:
 0.0+0.0im  1.0+0.0im  3.0+0.0im  4.0+0.0im  3.0+0.0im
1.0+0.0im
 1.0+0.0im  2.0+0.0im  4.0+0.0im  5.0+0.0im  4.0+0.0im
2.0+0.0im
 3.0+0.0im  4.0+0.0im  6.0+0.0im  7.0+0.0im  6.0+0.0im
4.0+0.0im
 4.0+0.0im  5.0+0.0im  7.0+0.0im  8.0+0.0im  7.0+0.0im
5.0+0.0im
 3.0+0.0im  4.0+0.0im  6.0+0.0im  7.0+0.0im  6.0+0.0im
4.0+0.0im
```

```
 1.0+0.0im  2.0+0.0im  4.0+0.0im  5.0+0.0im  4.0+0.0im
2.0+0.0im
```

- now, we **evaluate**

$$Q\Lambda Q^H u$$

- the **diagonal** matrix just **scales** each **element** of the **vector**
  both are packed the same way as $2D$ arrays
- the **last** product is the **inverse** DFT
- because the **initial vector** was **simple**, the **convolution** just **inserts** the **stencil**
- because of the **three** FFT calls, the **total cost** is

$$O\left(MN\left(log(M) + log(N)\right)\right)$$

```
julia> first = L.*fft(u);

julia> sol = ifft(first)
6×6 Matrix{ComplexF64}:
 0.0+0.0im   0.0+0.0im   0.0+0.0im   0.0+0.0im   0.0+0.0im
0.0+0.0im
 0.0+0.0im   0.0+0.0im   0.0+0.0im   0.0+0.0im  -1.0+0.0im
0.0+0.0im
 1.23358e-16+0.0im  -2.46716e-17+0.0im  -2.46716e-17+0.0im
-2.46716e-17+0.0im
 3.70074e-17+0.0im   1.23358e-17+0.0im  -3.70074e-17+0.0im
1.23358e-17+0.0im

julia> real(sol)
6×6 Matrix{Float64}:
 0.0   0.0   0.0   0.0   0.0   0.0
 0.0   0.0   0.0  -1.0   0.0   0.0
 0.0   0.0   0.0  -4.0  -1.0   0.0
 0.0   0.0   0.0  -1.0   0.0   0.0
 1.23358e-16  -2.46716e-17  -2.46716e-17   1.23358e-16  -
2.46716e-17  -2.46716e-17
 3.70074e-17   1.23358e-17  -3.70074e-17   1.23358e-17  -
3.70074e-17   1.23358e-17
```

Convolution

- this method is used in **codes** that **compute** a **convolution**
- here is a **library function** that takes in a **convolution kernel** as a 5×5 ***matrix***
  the array is padded with zeros

```
julia> using DSP

julia> A = zeros(5,5); A[3,2] = 1; A[5,5] = 1;

julia> A
5×5 Matrix{Float64}:
 0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0
 0.0  1.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  0.0
 0.0  0.0  0.0  0.0  1.0

julia> G = [1 2 3; 4 5 6; 7 8 9];

julia> Out = conv(A,G)
7×7 Matrix{Float64}:
 -4.45009e-16   0.0           1.45009e-15   8.70052e-16
0.0           0.0           2.90017e-16
  4.6693e-16    7.25044e-16   5.80035e-16   5.07531e-16  -
5.80035e-16  -5.80035e-16   0.0
  7.43918e-17   1.0           2.0           3.0          -
5.80035e-16   5.80035e-16   5.80035e-16
 -4.6693e-16    4.0           5.0           6.0          -
7.25044e-17   7.25044e-16   5.80035e-16
 -3.5316e-16    7.0           8.0           9.0
1.0           2.0           3.0
  1.37898e-16   4.35026e-16   4.35026e-16   8.70052e-16
4.0           5.0           6.0
  3.0532e-16    5.80035e-16   2.90017e-16   1.37758e-15
7.0           8.0           9.0
```

De-Convolution

- where things become very interesting is for deconvolving

$$-u_{i,j-1} - u_{i-1,j} + 4u_{i,j} - u_{i+1,j} - u_{i,j+1} = -h^2 f_{i,j}$$

- means: find u such that when you convolve it with the stencil you get the right-hand side

$$Q\Lambda Q^H u = b$$

$$u = (Q\Lambda Q^H)^{-1} b = Q\Lambda^{-1} Q^H b$$

- this needs a small tweak, since for this operator the matrix is not invertible
  one of the eigenvalues is 0
- that means that $fft(b)$ must have a $zero$ in the corresponding term
  note that this solves the problem on a periodic domain

```
julia> L = fft(s)

julia> Linv = 1.0 ./ L; Linv[1,1] = 0;

julia> u = real(ifft( fft(b) .* Linv ));
```

Computing the SVD

SVD Factorization

- **limitations** of eigen decomposition
  - only **valid** for **square** matrices
  - for **some** matrices, we **must** use **complex numbers** to diagonalize
- **SVD fixes** all of that
  - for a **symmetric** matrix

$$A = Q\Lambda Q^T$$

Q: orthogonal, square
Λ: diagonal, real

  - for any **matrix**

$$A = U\Sigma V^T$$

U, V: orthogonal; only square if A is square
Σ: diagonal, real, non-negative

# of rows in A = # rows in U
# of cols in A = # cols in V^T

How to Compute

- take a **tall** matrix $m \geq n$

$$A \in \mathbb{R}^{m \times n}$$

- start by **diagonalizing** the $n \times n$ symmetric **matrix**

$$A^T A = V \Lambda V^T$$

- the **columns** of V form an **orthonormal basis** for the $n - dimensional$ space

- apply $\boldsymbol{A}$ to each **vector**

$$(\boldsymbol{Av_i}) \cdot (\boldsymbol{Av_j}) = (\boldsymbol{Av_i})^T \boldsymbol{Av_j} = \boldsymbol{v_i^T A^T Av_j} = \boldsymbol{v_i^T}(\boldsymbol{\lambda_j v_j}) = \boldsymbol{\lambda_j}(\boldsymbol{v_i \cdot v_j})$$

- if $\boldsymbol{i = j}$ it shows that the **eigenvalue** is **non-negative**

$$\boldsymbol{0 \leq \|Av_i\|^2 = \lambda_j \|v_j\|^2 = \lambda_j}$$

- the **lengths** of these **vectors** are called the $\boldsymbol{singular\ values}$

$$\boldsymbol{\sigma_i = \|Av_i\| = \sqrt{\lambda_i}}$$

- **sorting** the values from **largest** to **smallest** meant that the tail could be $\boldsymbol{0}$
- let's say that the **first** $\boldsymbol{r \leq n}$ are $\boldsymbol{non-zero}$, then $\boldsymbol{n - r\ zeros}$

$$\boldsymbol{AV = A[v^1 v^2 \dots v_n] = [Av^1 Av^2 \dots Av_r 0 \dots 0]}$$

- the **first** $\boldsymbol{r}$ vectors are $\boldsymbol{non-zero}$, and what we just showed is that they are **orthogonal**

$$\boldsymbol{(Av_i) \cdot (Av_j) = \lambda_j (v_i \cdot v_j) = 0, i \neq j}$$

- which means that the **following vectors** are **orthonormal**

$$\boldsymbol{u_i = Av_i / \|Av_i\| = Av_i / \sigma_i, i = 1, \dots, r}$$

- we **create** $\boldsymbol{n - r}$ more **vectors** that are all **orthonormal**
- we could even **create** $\boldsymbol{m - r}$ more ($\boldsymbol{m \geq n}$)
  but these have no numerical use

$$\boldsymbol{AV = A[v^1 v^2 \dots v_n] = [\sigma^1 u^1 \sigma^2 u^2 \dots \sigma_n u_n] = [u^1 u^2 \dots u_n]\Sigma = U\Sigma}$$

$$\boldsymbol{Av_i = \sigma_i u_i}$$

- $\boldsymbol{V}$ is **invertible** and **orthogonal** so,

$$\boldsymbol{A = U\Sigma V^T}$$

Preferred Method (Julia)

- start with a 4×3 matrix A
- dedicated **SVD** routines do a **faster** and more **accurate** job
- Julia **computes** the **thin form**, so U is 4×3

1

- the **singular values** are **not** moved to **0**
  clearly more accurate than the previous method

2

- the V matrix is stored in terms of the **transpose, not** V

3

```
julia> A
4×3 Matrix{Float64}:
  2.64434   2.77354   0.87934
 -0.114416  0.854368  0.645425
 -0.763388  4.9565    1.01532
 -0.0203128 -2.02325  -1.04917

julia> f = svd(A)
SVD{Float64, Float64, Matrix{Float64}, Vector{Float64}}
U factor:
4×3 Matrix{Float64}:
 -0.459242  -0.152173   0.827237
  0.153632  -0.553737   0.0936324
 -0.806229   0.480617  -0.285158
 -0.348253   0.662741   0.474971

singular values:
3-element Vector{Float64}:
 6.35597336610859
 0.8198324736388264
 2.759682200799747e-16

Vt factor:
3×3 Matrix{Float64}:
  0.111627  -0.957811  -0.264837
 -0.49214    0.178244  -0.852072
 -0.863329   0.225451  -0.45148
```

1

2

3

Special Case:

$m < n$

- for **broad matrices** (row < column) the **transpose** is **tall**, so take the **SVD** of that and **transpose** it **back**

$$A^T = U \Sigma V^T$$

$$A = V \Sigma U^T$$

```
julia> f = svd(A)
SVD{Float64, Float64, Matrix{Float64}, Vector{Float64}}
U factor:
4×3 Matrix{Float64}:
 -0.459242  -0.152173   0.827237
  0.153632  -0.553737   0.0936324
 -0.806229   0.480617  -0.285158
 -0.348253   0.662741   0.474971

singular values:
3-element Vector{Float64}:
 6.35597336610859
 0.8198324736388264
 2.759682200799747e-16

Vt factor:
3×3 Matrix{Float64}:
  0.111627  -0.957811  -0.264837
 -0.49214    0.178244  -0.852072
 -0.863329   0.225451  -0.45148

julia> ft = svd(A')
SVD{Float64, Float64, Adjoint{Float64, Matrix{Float64}},
Vector{Float64}}
U factor:
3×3 adjoint(::Matrix{Float64}) with eltype Float64:
  0.111627  -0.49214   0.863329
 -0.957811   0.178244  0.225451
 -0.264837  -0.852072 -0.45148

singular values:
3-element Vector{Float64}:
 6.35597336610859
 0.8198324736388264
 2.759682200799747e-16

Vt factor:
3×4 adjoint(::Matrix{Float64}) with eltype Float64:
 -0.459242   0.153632  -0.806229   0.348253
 -0.152173  -0.553737   0.480617   0.662741
  0.827237   0.0936324 -0.285158   0.47497
```

same matrices transposed

same matrices transposed

adjoint means conjugate transposed